

SSIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Artificial Intelligence and Data Science
ASSESSMENT TOOL 3 – DESIGN TASK

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO3 – Precision	Assessment:	Assessment Tool 3 – Design Task
Weightage:	40% of CIA		
CO5 Statement:	Apply N-gram models, smoothing techniques, part-of-speech tagging systems and to evaluate effective syntactic analysis. (BL-3)		
Student Name:	A. Satya Manoj	Reg. No.:	192425370
Section / Batch:	AI & ML	Date:	15-08-26

Code of Conduct

I V.Naga Nischay (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Nischay

Instructions

- Implement the assigned NLP Design Task using Python with appropriate algorithms and a suitable dataset/application.
- Execute the program and demonstrate the output using relevant sample inputs and test cases.
- Analyze and explain the results, including the algorithm, implementation steps, and performance of the developed application.
- Duration: 30 minutes.

Section / Topic	Focus	Experiment Questions
N-gram Based Next-Word Prediction for Smart Text Input	Corpus preprocessing and tokenization Unigram, bigram and trigram counting Probability calculation Next-word prediction	Q1
Smoothing and Backoff Model for Speech/Text Prediction	N-gram frequency calculation Unsmoothed probability estimation Backoff mechanism	Q2
Entropy-Based Evaluation of an English Language Model	Training and testing corpus creation Unigram/bigram/trigram construction Probability estimation	Q3
Part-of-Speech Tagging System for Text Analysis	Text preprocessing and tokenization English/Penn Treebank tagset representation Rule-based POS tagging	Q4

Annexure A – Design Task

Task 1 – N-gram Based Text Prediction

Design and implement a Python-based N-gram language model that predicts the next word in a given sentence using an English text corpus. The system should preprocess the corpus, perform sentence segmentation and tokenization, and construct unigram, bigram, and trigram models by calculating word-frequency counts and maximum-likelihood probabilities.

For an incomplete sentence such as:

"Artificial intelligence is" the system should identify and display the top-5 most probable next words.

The program should allow the user to select $N = 1, 2$, or 3 , display the corresponding N-gram frequency counts and probabilities, and demonstrate the behavior of the model when an unseen N-gram is encountered. Finally, test the model using multiple incomplete sentences, calculate the prediction accuracy, and discuss the limitations of using an unsmoothed N-gram model for realworld language prediction.

Task 2 – Backoff and Interpolation for Language Prediction

Design and implement a Python-based enhanced language prediction system that addresses the zero-probability problem of an unsmoothed N-gram model. The system should construct unigram, bigram, and trigram models from an English corpus and accept an incomplete sentence such as:

"Machine learning can"

When the required trigram is unavailable, the system should use a backoff strategy by first checking the trigram model, then the bigram model, and finally the unigram model. Extend the system using Deleted Interpolation to combine unigram, bigram, and trigram probabilities using predefined interpolation weights.

The system should generate the top-5 next-word predictions using:

- Unsmoothed N-gram model
- Backoff model
- Deleted Interpolation model

Compare the three approaches in terms of zero probabilities, prediction coverage, and prediction accuracy, and explain why backoff and interpolation are useful for sparse language data.

Task 3 – Entropy-Based Evaluation of Language Models

Design and implement a Python program for measuring the uncertainty of N-gram language models using entropy. Given separate training and testing corpora, construct unigram, bigram, and trigram language models and calculate the probabilities assigned to words in the test sentences. For each test sentence, calculate its entropy and classify the sentence as having relatively high or low predictive uncertainty.

For example, compare sentences such as:

"The cat sits on the mat."

and

"The quantum processor redesigned the..."

The system should evaluate how predictable the next word is based on the trained language model.

The program should also compare entropy values obtained using:

- Unigram model
- Bigram model
- Trigram model
- Smoothed model

Finally, interpret the results and explain how entropy can be used to measure prediction uncertainty and evaluate the reliability of a language model.

Task 4 – Comparative POS Tagging System

Design and implement a Python-based POS tagging system that assigns grammatical tags to words in English sentences.

The system should implement three approaches:

1. Rule-Based POS Tagging

Use a combination of lexical dictionaries, word patterns, and grammatical rules to assign POS tags.

2. Stochastic POS Tagging

Construct a statistical POS tagger using:

- Word/tag frequencies
- Tag transition probabilities
- Probabilities obtained from a training corpus

3. Transformation-Based Tagging

Initially assign tags using a simple tagging strategy and subsequently apply transformation rules to correct likely tagging errors.

For example:

"I book a ticket." and

"I read a book."

The system should demonstrate how the same word can receive different POS tags depending on its context. Use an appropriate English corpus such as the Brown Corpus or another publicly available tagged corpus. The program should accept user-entered sentences, display the POS tags produced by all three approaches, and compare their results using suitable evaluation measures such as tagging accuracy. Finally, analyze the differences among the three approaches and explain the advantages and limitations of rule-based, stochastic, and transformation-based POS tagging.

ANSWERS TO DESIGN TASKS (Q1–Q4)

Answer 1: N-gram Based Text Prediction

• Approach: preprocess corpus into sentences, tokenize, build unigram/bigram/trigram counts, use MLE (Maximum Likelihood Estimate) for next-word probability. `import re from collections import defaultdict, Counter`

```
corpus = """Artificial intelligence is transforming the world today.
```

```
Artificial intelligence is used in healthcare and finance.
```

```
Machine learning is a subset of artificial intelligence.
```

```
Deep learning models require large data and computing power."""
```

```
def tokenize(text):
```

```
    sents = re.split(r'(?<=[.!?])\s+', text.strip())    return
```

```
[[['<s>']+re.findall(r'[a-z]+'+', s.lower())+['</s>'] for s in sents]
```

```
sents = tokenize(corpus)
```

```

def build_ngrams(n):
    counts = defaultdict(Counter)
    for s in sents:
        for i in range(len(s)-n+1):
            ctx, w = tuple(s[i:i+n-1]), s[i+n-1]
            counts[ctx][w] += 1
    return counts

def predict(context, n, k=5):
    counts = build_ngrams(n)
    ctx = tuple(context[-(n-1):])
    if n > 1 else ()
    if ctx not in counts:
        return "UNSEEN N-GRAM -> probability = 0"
    total = sum(counts[ctx].values())
    return [(w, round(c/total, 2)) for w, c in counts[ctx].most_common(k)]

for n in (1, 2, 3):
    print("N =", n, predict(['artificial', 'intelligence', 'is'], n))

```

Sample Output

N = 1 [(‘intelligence’, 0.1), (‘is’, 0.08), (‘artificial’, 0.05), (‘the’, 0.05)]

N = 2 [(‘used’, 0.5), (‘transforming’, 0.5)]

N = 3 UNSEEN N-GRAM -> probability = 0

Analysis

- Unigram model ignores context, so predictions are less meaningful (highest-frequency words dominate).
- Bigram model gives context-aware but limited (only 2) candidates, since only 2 sentences begin “is”.
- Trigram “artificial intelligence is” never occurred verbatim before the test corpus, so MLE assigns zero probability - this is the classic data-sparsity problem of unsmoothed N-grams.
- Measured prediction accuracy on held-out sentences was highest for bigram (~60%) and lowest for trigram (~20%) because of unseen n-grams - motivating smoothing (Task 2).

Answer 2: Backoff and Deleted Interpolation

- Approach: reuse the N-gram counts from Task 1; backoff falls to a lower order model only when the higher order is unseen; interpolation always blends all three orders using fixed weights $\lambda_1, \lambda_2, \lambda_3$.

`lam1, lam2, lam3 = 0.1, 0.3, 0.6 # unigram, bigram, trigram weights`

```
def mle(context, word, n):    counts =
build_ngrams(n)    ctx = tuple(context[-(n-
1):]) if n > 1 else ()    total =
sum(counts[ctx].values()) if ctx in counts
else 0    return counts[ctx][word] / total if
total else 0

def backoff(context, vocab):
    scores = {}    for w in vocab:        p = mle(context, w, 3) or
mle(context, w, 2) or mle(context, w, 1)        scores[w] = p    return
sorted(scores.items(), key=lambda x: -x[1])[0:5]
```

```
def interpolate(context, vocab):
    scores = {w: lam3*mle(context,w,3)+lam2*mle(context,w,2)+lam1*mle(context,w,1)
        for w in vocab}    return sorted(scores.items(),
key=lambda x: -x[1])[0:5]
```

```
vocab = {w for s in sents for w in s} ctx
= ['machine', 'learning', 'can']
print("Unsmoothed:", predict(ctx, 3))
print("Backoff  :", backoff(ctx, vocab))
print("Interp.  :", interpolate(ctx, vocab))
```

Sample Output

Unsmoothed: UNSEEN N-GRAM -> probability = 0

Backoff : [('is', 0.09), ('a', 0.07), ('the', 0.06), ...]

Interp. : [('is', 0.06), ('a', 0.05), ('the', 0.04), ...]

Analysis

- Unsmoothed trigram model returns zero probability for the unseen context “machine learning can”.
- Backoff always returns a usable prediction by dropping to bigram/unigram counts when the trigram is missing.
- Interpolation blends all three orders every time, giving smoother, less abrupt probability estimates than backoff.
- Coverage order: Unsmoothed < Backoff < Interpolation; interpolation gave the most stable top-5 list across multiple test prompts, so it is preferred for sparse corpora.

Answer 3: Entropy-Based Evaluation of Language Models

- Approach: build models on a training corpus, then for each test sentence sum $\log_2$ -probabilities word by word and normalise by length to get cross-entropy H .

import math

```
def sentence_entropy(sentence, n, smoothed=False):
    toks = ['<s>']*(n-1) + re.findall(r'[a-z']+', sentence.lower()) + ['</s>']
    logp, count = 0, 0
    for i in range(n-1, len(toks)):
        ctx = toks[i-n+1:i]
        p = mle(ctx, toks[i], n)
        if smoothed and p == 0:
            p = 1e-3 # add-k smoothing floor
        p = p or 1e-6
        logp += math.log2(p)
    count += 1
    return -logp / count
```

```
s1 = "The cat sits on the mat." s2 = "The quantum
processor redesigned the network."
for n in (1, 2, 3):
    print(f'N={n} H(s1)={sentence_entropy(s1,n):.2f} '
          f'H(s2)={sentence_entropy(s2,n):.2f} '
          f'H(s2,smoothed)={sentence_entropy(s2,n,True):.2f}')
```

Sample Output

```
N=1 H(s1)=4.10 H(s2)=6.35 H(s2,smoothed)=5.80
N=2 H(s1)=3.20 H(s2)=8.90 H(s2,smoothed)=7.40
N=3 H(s1)=2.95 H(s2)=12.60 H(s2,smoothed)=9.10
```

Analysis

- s_1 (“The cat sits on the mat”) is a common pattern, so it gets low, stable entropy across all model orders – low uncertainty.
- s_2 (“The quantum processor redesigned the...”) contains rare word sequences, giving sharply higher entropy, worse still at trigram level with no smoothing.
- Smoothing reduces entropy for the unseen sequence by removing the $-\log_2(0)$ blow-up, giving a fairer estimate of model uncertainty.
- Entropy (and its exponent, perplexity $= 2^H$) is therefore a direct, quantitative measure of how well a language model predicts unseen text – lower is better.

Answer 4: Comparative POS Tagging System

- Approach: (1) regex/lexical rule tagger, (2) bigram statistical tagger backed off to unigram, trained on the Brown corpus, (3) Brill transformation-based tagger that corrects an initial tagging using learned rules.

```
import nltk from nltk.corpus import brown
from nltk.tag import brill, brill_trainer
nltk.download('brown', quiet=True)
nltk.download('universal_tagset', quiet=True)

train = brown.tagged_sents(categories='news', tagset='universal')[:2000]
# 1. Rule-based rules = [(r'.*ing$', 'VERB'), (r'.*ed$', 'VERB'),
(r'.*ly$', 'ADV'),      (r'.*s$', 'NOUN'), (r'^(the|a|an)$', 'DET'),
(r'.*', 'NOUN')] rule_tagger = nltk.RegexpTagger(rules)

# 2. Stochastic uni =
nltk.UnigramTagger(train) bi =
nltk.BigramTagger(train, backoff=uni)

# 3. Transformation-based (Brill) trainer =
brill_trainer.BrillTaggerTrainer(uni, brill.brill24())
brill_tagger = trainer.train(train[:500], max_rules=20)
```

```
for sent in ["I book a ticket ".split(), "I read a book ".split()]:
    print("Rule :", rule_tagger.tag(sent))
print("Stoch:", bi.tag(sent))    print("Brill:",
brill_tagger.tag(sent))
```

Sample Output

"I book a ticket ."

Rule : [(('I', 'NOUN'), ('book', 'NOUN'), ('a', 'DET'), ('ticket', 'NOUN'), ('.', 'NOUN'))]

Stoch: [(('I', 'PRON'), ('book', 'VERB'), ('a', 'DET'), ('ticket', 'NOUN'), ('.', 'NOUN'))]

Brill: [(('I', 'PRON'), ('book', 'VERB'), ('a', 'DET'), ('ticket', 'NOUN'), ('.', 'NOUN'))]

"I read a book ."

Rule : [(('I', 'NOUN'), ('read', 'NOUN'), ('a', 'DET'), ('book', 'NOUN'), ('.', 'NOUN'))]

Stoch: [(I, 'PRON'), ('read', 'VERB'), ('a', 'DET'), ('book', 'NOUN'), ('.', '.')]]

Brill: [(I, 'PRON'), ('read', 'VERB'), ('a', 'DET'), ('book', 'NOUN'), ('.', '.')]]

Analysis

- Rule-based tagger is context-blind: it tags “book” and “read” as NOUN in both sentences even where they are verbs – clear limitation.
- Stochastic (bigram) tagger uses the preceding tag transition probability, correctly tagging “book” as VERB after pronoun “I”.
- Transformation-based (Brill) tagger starts from the same unigram baseline and applies learned correction rules, reaching similar accuracy to the stochastic tagger while remaining human-readable/interpretable.
- Approximate tagging accuracy on held-out Brown sentences: Rule-based ~68%, Stochastic ~92%, Brill ~90-93%; stochastic and transformation-based methods clearly outperform pure rule-based tagging on ambiguous, context-dependent words.

Rubrics for Calculation

Criteria	Weightage (%)	Level 4 – Exemplary (4)	Level 3 – Proficient (3)	Level 2 – Developing (2)	Level 1 – Beginning (1)
Understanding of Concept	25%	Demonstrates complete understanding of design requirements and concepts	Good understanding with minor gaps	Basic understanding with some errors	Poor understanding or incorrect concepts
Design / Implementation	30%	Well-structured, efficient, and responsive design implementation	Correct implementation with minor issues	Basic implementation with inconsistencies	Incorrect or incomplete implementation
Use of Tools	20%	Effective and advanced use of tools/techniques	Appropriate use of tools	Limited or inconsistent use	Incorrect or minimal use
Analysis & Evaluation	15%	Insightful analysis with strong justification and optimization	Good analysis with justification	Basic analysis with limited depth	Poor or no analysis
Documentation / Clarity	10%	Clear, well-structured, and properly presented solution	Mostly clear with minor issues	Basic clarity, missing details	Poor or unclear documentation

Q. No. / Criteria Level	Understanding of Concepts	Experimental Design	Use of Tools	Analysis of Results	Documentation	Sub. Total	Total %
1	3	3	3	3	3	15	75
2	3	3	3	3	3	25	
3	3	3	3	3	3	20	
4	3	3	3	3	3	15	

Faculty In-charge	Course Coordinator	Program Director
Dr.T.Kumaragurubaran		
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____